

```

        if (head->value == num)
            delete head;
        else
        {
            // Initialize nodePtr to head of list.
            nodePtr = head;

            // Skip all nodes whose value member is
            // not equal to num.
            while (nodePtr->value != num)
            {
                previousNode = nodePtr;
                nodePtr = nodePtr->next;
            }
            // Link the previous node to the node after
            // nodePtr, then delete nodePtr.
            previousNode->next = nodePtr->next;
            delete nodePtr;
        }
    }
}

35. NumberList::~NumberList()
{
    ListNode *nodePtr, *nextNode;
    nodePtr = head;
    while (nodePtr != nullptr)
    {
        nextNode = nodePtr->next;
        nodePtr->next = nullptr;
        nodePtr = nextNode;
    }
}

```

Programming Challenges

1. Your Own Linked List

Design your own linked list class to hold a series of integers. The class should have member functions for appending, inserting, and deleting nodes. Don't forget to add a destructor that destroys the list. Demonstrate the class with a driver program.

2. List Print

Modify the linked list class you created in Programming Challenge 1 to add a `print` member function. The function should display all the values in the linked list. Test the class by starting with an empty list, adding some elements, and then printing the resulting list out.

3. List Copy Constructor

Modify your linked list class of Programming Challenges 1 and 2 to add a copy constructor. Test your class by making a list, making a copy of the list, and then displaying the values in the copy.

4. List Reverse

Modify the linked list class you created in the previous programming challenges by adding a member function named `reverse` that rearranges the nodes in the list so that their order is reversed. Demonstrate the function in a simple driver program.

5. List Search

Modify the linked list class you created in the previous programming challenges to include a member function named `search` that returns the position of a specific value in the linked list. The first node in the list is at position 0, the second node is at position 1, and so on. If `x` is not found on the list, the search should return `-1`. Test the new member function using an appropriate driver program.

6. Member Insertion by Position

Modify the list class you created in the previous programming challenges by adding a member function for inserting a new item at a specified position. A position of 0 means that the value will become the first item on the list, a position of 1 means that the value will become the second item on the list, and so on. A position equal to or greater than the length of the list means that the value is placed at the end of the list.

7. Member Removal by Position

Modify the list class you created in the previous programming challenges by adding a member function for deleting a node at a specified position. A value of 0 for the position means that the first node in the list (the current head) is deleted. The function does nothing if the specified position is greater than or equal to the length of the list.

8. List Template

Create a list class template based on the list class you created in the previous programming challenges.

9. Rainfall Statistics Modification

Modify Programming Challenge 2 in Chapter 7 (Rainfall Statistics) to let the user decide how many months of data will be entered. Use a linked list instead of an array to hold the monthly data.

10. Payroll Modification

Modify Programming Challenge 10 in Chapter 7 (Payroll) to use three linked lists instead of three arrays to hold the employee IDs, hours worked, and wages. When the program starts, it should ask the user to enter the employee IDs. There should be no limit on the number of IDs the user can enter.

11. List Search

Modify the `LinkedList` template shown in this chapter to include a member function named `search`. The function should search the list for a specified value. If the value is found, it should return a number indicating its position in the list. (The first node is node 1, the second node is node 2, and so forth.) If the value is not found, the function should return 0. Demonstrate the function in a driver program.



12. Double Merge

Modify the `NumberList` class shown in this chapter to include a member function named `mergeArray`. The `mergeArray` function should take an array of `doubles` as its first argument and an integer as its second argument. (The second argument will specify the size of the array being passed into the first argument.)

The function should merge the values in the array into the linked list. The value in each element of the array should be inserted (not appended) into the linked list. When the values are inserted, they should be in numerical order. Demonstrate the function with a driver program. When you are satisfied with the function, incorporate it into the `LinkedList` template.

13. Rainfall Statistics Modification #2

Modify the program you wrote for Programming Challenge 9 so that it saves the data in the linked list to a file. Write a second program that reads the data from the file into a linked list and displays it on the screen.

14. Overloaded `[]` Operator

Modify the linked list class that you created in Programming Challenge 1 (or the `LinkedList` template presented in this chapter) by adding an overloaded `[]` operator function. This will give the linked list the ability to access nodes using a subscript, like an array. The subscript 0 will reference the first node in the list, the subscript 1 will reference the second node in the list, and so forth. The subscript of the last node will be the number of nodes minus 1. If an invalid subscript is used, the function should throw an exception.

15. `pop` and `push` Member Functions

The STL list container has member functions named `pop_back`, `pop_front`, `push_back`, and `push_front`, as described in Table 17-1. Modify the linked list class that you created in Programming Challenge 1 (or the `LinkedList` template presented in this chapter) by adding your own versions of these member functions.